

自进化机制

Hermes Agent自进化机制

目录

- [项目概述](#)
- [自进化架构总览](#)
 - [自进化反馈回路](#)
- [核心自进化机制详解](#)
 - [3.1 技能创建与自维护](#)
 - [3.2 记忆持久化系统](#)
 - [3.3 跨会话检索与模式识别](#)
 - [3.4 上下文压缩与信息保护](#)
 - [3.5 RL训练数据管道](#)
- [辅助进化子系统](#)
 - [4.1 用户建模 \(Honcho\)](#)
 - [4.2 多实例Profile隔离](#)
 - [4.3 MCP动态工具发现](#)
- [工具层自进化支持](#)
- [轨迹工程：数据驱动的进化](#)
- [进化机制关联图](#)
- [技术实现细节](#)
 - [8.1 技能激活机制](#)
 - [8.2 记忆注入策略](#)
 - [8.3 安全机制](#)
 - [8.4 平台适配层](#)
- [总结与评价](#)
 - [9.1 核心设计哲学](#)
 - [9.2 进化机制分类](#)
 - [9.3 创新点](#)
 - [9.4 反馈信号体系全景分析](#)
 - [9.5 局限性](#)
- [参考文件索引](#)

1. 项目概述

Hermes Agent 是由 [Nous Research](#) 开发的自进化（Self-Improving）AI Agent，定位为"通用、自进化、跨平台"的AI助手。其核心理念在于：

"创建技能（Skills）从经验中学习，在使用中不断改进，运行在任何地方。"

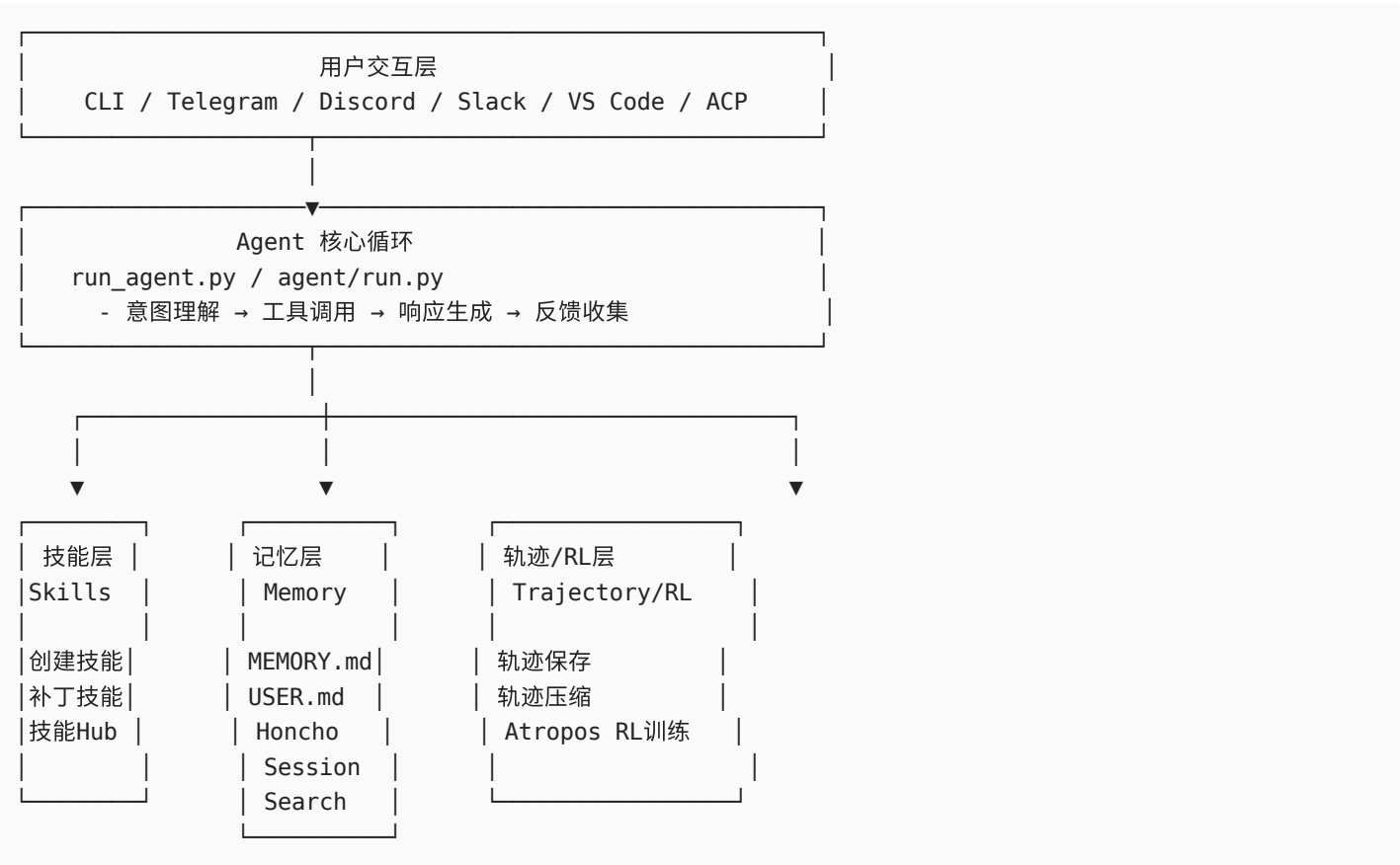
关键特性

特性	描述
内置学习循环	从复杂任务、错误纠正、非平凡工作流程中发现并固化知识

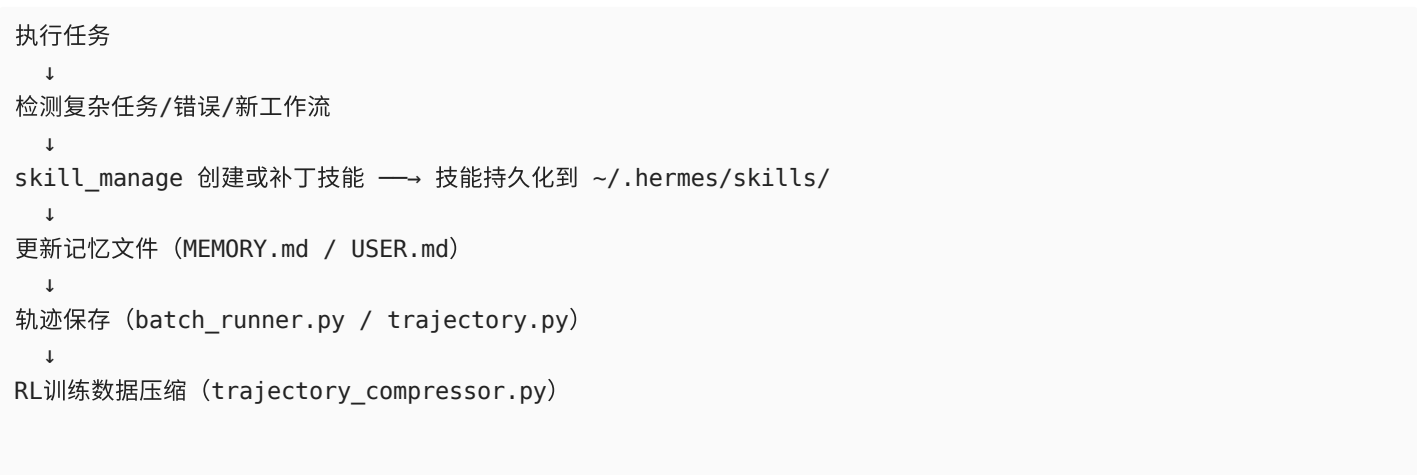
特性	描述
记忆系统	基于文件的持久化记忆（MEMORY.md / USER.md）+ Honcho用户建模
跨会话检索	FTS5全文搜索 + LLM摘要压缩，支持跨越多个会话的上下文复用
技能管理	Agent可自主创建、编辑、补丁化技能，运行时自我维护
RL训练支持	轨迹保存、压缩、与Tinker-Atropos RL环境集成
多平台网关	Telegram、Discord、Slack、WhatsApp、Signal等消息平台
MCP集成	动态发现和调用MCP服务器提供的外部工具

2. 自进化架构总览

Hermes Agent 的自进化并非通过修改自身代码实现，而是通过**多层持久化知识管理系统**形成闭环：



自进化反馈回路



↓
下一会话：检索历史技能/记忆/会话 → 复用经验

📌 Note

反馈信号是自进化闭环的关键：

- Agent的自进化本质上是一个"感知-反馈-调整"的闭环过程。无论是技能创建、记忆更新还是轨迹压缩，都依赖明确的反馈信号来驱动：任务成功了吗？用户满意吗？与历史方案相比有何改进？
- 这与强化学习（RL）的设计哲学高度一致：外部环境给出奖励/惩罚，智能体据此调整策略。区别在于，Agent的反馈信号来源更为多样，不仅来自任务成功率，还包括用户显性纠正、工具调用异常、上下文压缩损失评估等多维度信息。

3. 核心自进化机制详解

3.1 技能创建与自维护

3.1.1 技能管理工具

文件: `tools/skill_manager_tool.py`

Agent通过 `skill_manage` 工具对技能进行完整的CRUD (Create, Read, Update, Delete)操作：

```
# 工具支持的 action 参数
CREATE    # 创建新技能（从成功经验中提取）
EDIT      # 编辑现有技能的完整内容
PATCH    # 小范围补丁（快速修复错误/遗漏）
DELETE    # 删除废弃技能
WRITE_FILE # 创建技能子文件（references、templates、assets）
REMOVE_FILE # 删除技能子文件
```

3.1.2 反馈信号与触发条件（设计理念）

📌 Note

核心设计洞察：隐式信号依赖是双刃剑

Hermes Agent 的技能触发机制构建在一套混合反馈信号体系之上，其核心矛盾在于：**最有价值的反馈信号（任务是否有价值、是否值得沉淀）恰恰是最难被量化的。**

创建技能的触发条件：

触发场景	示例
复杂任务（5+工具调用）成功完成	完成了跨多文件的重构任务
克服了棘手错误	发现并绕过了一个非显而易见的bug
用户纠正后方法奏效	用户指出正确方向后成功完成任务
发现非平凡工作流	找到了一条更优的CI/CD流水线

定量信号 vs 定性信号的分布：

触发条件	信号类型	可验证性	优势	风险
复杂任务（5+工具调用）	显式定量	高（精确计数）	可复现、无歧义	无法区分"复杂的无用功"和"复杂但有价值"
棘手错误被克服	隐式语义	低（依赖LLM判断）	能捕捉真正有价值的经验	可能过度创建技能（凡错即建）
非平凡 workflow 被发现	隐式语义	低	能捕捉无法预知的模式	判断主观，依赖模型自身元认知能力
用户纠正后方法奏效	隐式社交	中（需理解纠正意图）	用户参与降低了错误成本	过度依赖用户干预，削弱自主性

 Important

洞见：CREATE 触发本质上是"经验蒸馏"

- Agent 不仅执行任务，还在执行过程中持续评估"这个经验是否可泛化" -> 因为是自进化，所以还可能存在一个“经验过拟合”的问题
- 5+调用阈值降低了误判成本：如果一个 workflow 重复多次，每次都用到 5+ 工具，它很可能值得固化
- 这本质上是一个 **在线学习（Online Learning）** 的探索-利用权衡：Agent 边执行边决定是否将当前经验记入长期记忆

3.1.3 维护触发条件

Agent 被明确指示在以下情况下 **自动补丁技能**：

触发场景	维护动作
使用技能时遇到问题	立即 PATCH，不等待询问
指令过时或不完整	补全遗漏步骤和已知陷阱
OS 特定失败	添加平台相关补丁
发现新陷阱	在 SKILL.md 中添加警告

3.1.4 技能文件格式

技能采用 **YAML frontmatter + Markdown 正文** 的结构化格式：

```
---
name: git-interactive-rebase
description: 使用交互式rebase整理提交历史
version: 1.2.0
platforms: [macos, linux]
tags: [git, vcs, history]
author: hermes-agent # agent自我创建时标记
auto_updated: true # 标记为agent自维护
---

# Git 交互式Rebase技能

## 适用场景
当你需要整理提交历史、合并WIP提交、修改提交信息时使用。

## 使用步骤
```

1. 使用 `git log --oneline -n 20` 查看最近提交
2. 确定需要修改的提交数量 `n`
3. 执行 `git rebase -i HEAD~n`
4. 在编辑器中选择操作 (pick/reword/squash/fixup/drop)

常见陷阱

- ⚠️ 不要对已推送的提交执行rebase (除非确定没有协作)
- ⚠️ 处理冲突时, 按顺序解决, 不要跳过

Note

Anthropic提出的渐进式披露和懒加载流程

3.1.5 技能目录结构

```
~/hermes/skills/
├── git-interactive-rebase/
│   ├── SKILL.md           # 主技能文件
│   ├── references/        # 参考文档
│   │   └── git-rebase-demo.md
│   └── templates/         # 输出模板
│       └── commit-message-template.txt
├── docker-debugging/
│   └── SKILL.md
└── swe-expert/
    └── SKILL.md
```

3.1.6 技能注册到系统Prompt

文件: `agent/prompt_builder.py`

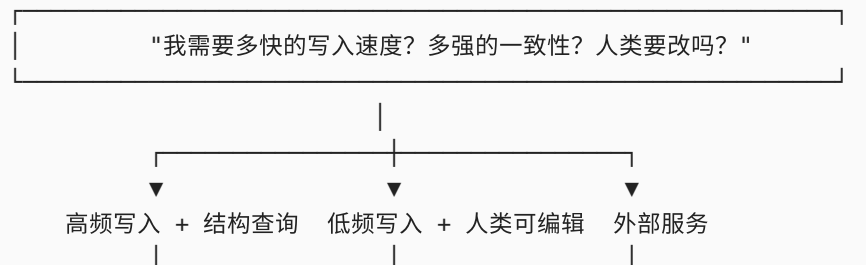
技能通过 `prompt_builder.py` 中的 `build_system_prompt()` 自动加载到Agent的系统Prompt中:

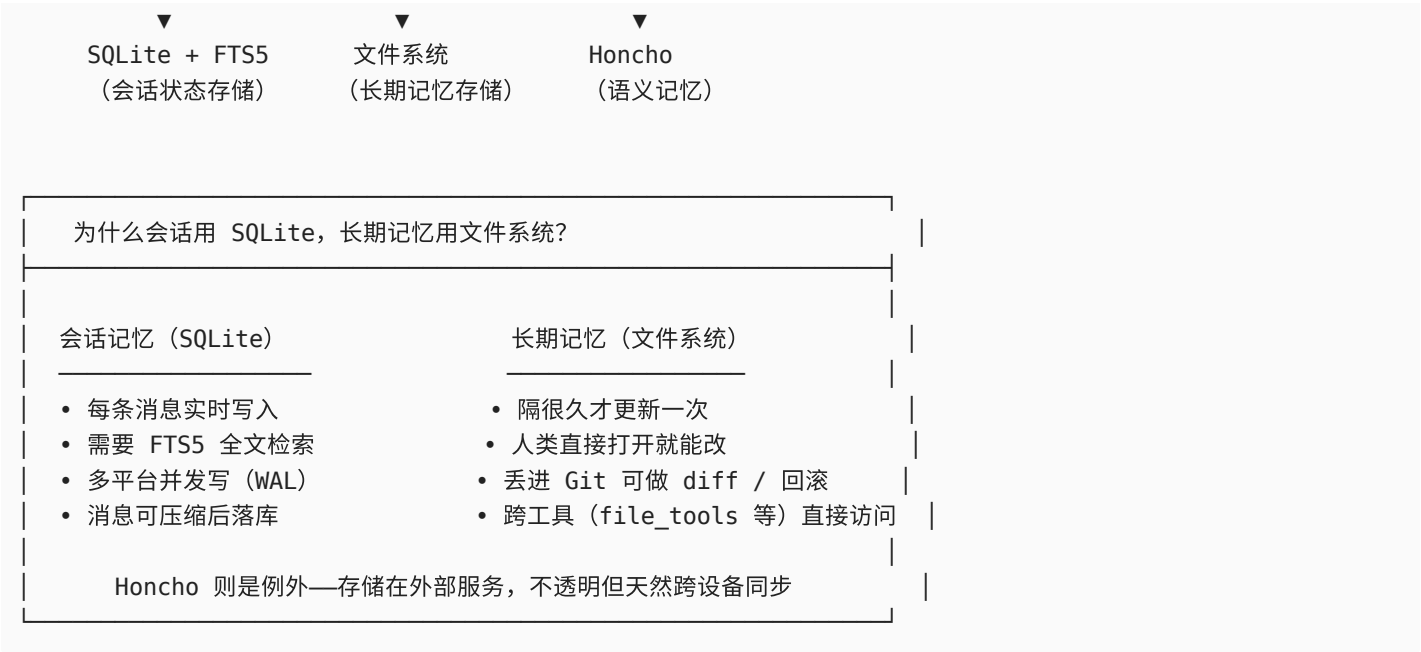
```
def build_system_prompt(identity, hints, skills, env_vars, soyl):
    # 1. 加载身份描述 (SOUL.md)
    # 2. 应用提示 (hints) — 包含技能使用指导
    # 3. 注入活跃技能列表及其描述
    # 4. 暴露环境变量
    # 5. 输出完整系统Prompt
```

3.2 记忆持久化系统

Hermes Agent 构建了三层记忆体系, 实现跨会话知识持久化。

3.2.0 存储介质分层设计





这形成了一个**一致性递降链**：SQLite > 文件系统 > Honcho。设计者的赌注是：访问频率越高的层，对一致性的要求也越高；访问频率越低的层，允许更大的延迟容忍度。

设计洞察三：人类可编辑性是"信任"的外化

文件系统作为 MEMORY.md / USER.md 的载体，不只是因为"方便"，而是因为它传递了一个设计哲学：**Agent 的知识不是 Agent 的私有财产，而是人类可以审查、干预、甚至推翻的。**

这与 RL 训练数据的黑箱性形成鲜明对比：

- 轨迹压缩后的训练数据：人类无法直接检查（需要解码 ShareGPT JSONL）
- MEMORY.md：任何时候打开文件都能读、修改、回滚

可以说，文件系统层是 Hermes Agent 对人类的"信任锚点"——无论 Agent 学到了什么、记住了什么，人类始终保有最终编辑权。这是一种**防止能力失控的制度性设计**。

设计洞察四：版本控制是"人类可编辑性"的延伸

将 MEMORY.md 放进 Git 的版本控制体系，意味着记忆具备了：

- **审计能力**（谁在什么时候改了什么）
- **回滚能力**（Agent 变笨了？一键恢复到上一个版本）
- **分支实验**（可以开一个 branch 测试不同的记忆策略）

这是 SQLite 会话层不具备的优势——你没法对 sessions.db 做 `git diff`。设计者显然认为：**长期知识值得版本控制，短期状态不值得。**

设计洞察五：Honcho 的"例外"揭示了平台级设计的权衡

Honcho 存储在外部服务而非本地，这是一个**有意为之的跨设备同步设计**。代价是：

- 离线不可用
- 数据不透明（看不到原始存储）
- 一致性依赖网络和第三方服务

设计者接受了这些代价，换取的是：**用户在任何设备上都能获得一致的用户画像**。这是一个典型的"C.A.P. 权衡"实践——在一致性和可用性之间，Honcho 选择的是可用性（弱一致性 + 跨设备共享）。

🔗 Important

总结：三层记忆是一套"信息生命周期"管理系统

存储介质选择 = $\text{argmin}(\text{访问频率失配成本} + \text{一致性失配成本} + \text{可编辑性失配成本})$

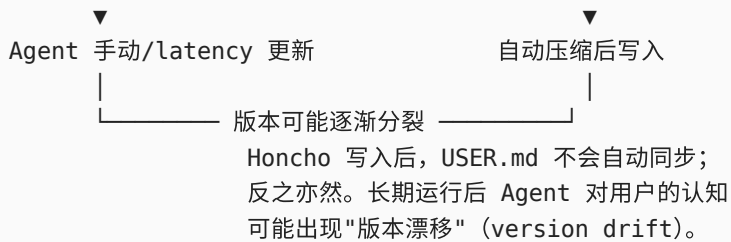
三层分别对应信息的三个生命周期阶段：

- **会话层 (SQLite)**：信息"热"的时候 → 高频写入、快速检索
- **长期记忆层 (文件系统)**：信息"温"的时候 → 人类介入、低频更新、可追溯
- **语义记忆层 (Honcho)**：信息"冷"的时候 → 跨设备共享、最终一致、模型蒸馏

没有任何一层是"最好"的存储，但组合在一起，刚好覆盖了 Agent 记忆的全生命周期需求。

跨层一致性的潜在风险（当前框架未覆盖）：

MEMORY.md (文件系统)	sessions.db (SQLite)



3.2.1 第一层：文件记忆 (MEMORY.md / USER.md)

文件: tools/memory_tool.py

MEMORY.md — Agent的私人笔记：

- 环境事实（项目结构、工具习惯、系统差异）
- 项目约定（代码风格、命名规范）
- 工具缺陷和已知问题
- 配置信息（不走弯路的关键配置）

USER.md — Agent对用户的认知：

- 用户角色和职责
- 偏好和沟通风格
- 期望和需求模式
- 协作习惯

3.2.2 第二层：会话状态 (SQLite + FTS5)

文件: hermes_state.py

```
# SQLite Schema
CREATE TABLE sessions (
    id TEXT PRIMARY KEY,           # 会话唯一标识 (UUID)
    source TEXT,                   # 'cli' | 'telegram' | 'discord' 等, 支持跨平台检索
    model TEXT,                    # 本会话使用的模型名称 (如 'claude-sonnet-4-6')
    created_at INTEGER,            # Unix 时间戳 (秒), 会话创建时间
    updated_at INTEGER,            # Unix 时间戳 (秒), 最近消息时间, 用于 LRU 清理
)

CREATE VIRTUAL TABLE sessions_fts USING fts5(
    # — FTS5 全文索引列 (content 实际存储在 sessions 表) —
    content,                       # 消息正文 (经过上下文压缩后的版本)
    role,                          # 消息角色: 'user' | 'assistant' | 'tool'
    session_id,                    # 外键关联 sessions.id, 支持按会话分组检索
    token_count,                   # 该消息的 token 数量, 用于排序和分页
    # — FTS5 内容同步配置 —
    content='sessions',            # 指向 sessions 表 (非独立存储), 避免数据冗余
    content_rowid='rowid',         # 通过 rowid 关联原始记录, 保证索引与表始终一致
);
```

特点：

- WAL模式：并发读 + 单一写
- FTS5全文索引：毫秒级跨会话搜索
- 按 source 标签区分会话来源

- 支持消息级Token计数

3.2.3 第三层：技能系统（持久化知识模块）

文件: `tools/skills_tool.py` / `tools/skill_manager_tool.py`

技能作为可执行、可补丁的持久化知识模块，比MEMORY.md更进一步：

- 有结构化的元数据（名称、版本、平台、标签）
- 可以包含子文件（references、templates、assets）
- 支持版本控制（显式version字段）
- 有激活/停用状态（可在config中开关）
- 有安全扫描机制（安装前校验）

 Note

技能 vs 记忆：何时沉淀到技能，何时沉淀到记忆？

代码中 `memory_tool.py` 的 schema description 给出了最直接的指引：

```
"Do NOT save task progress, session outcomes, completed-work logs, or temporary TODO state to memory; use session_search to recall those from past transcripts."
"If you've discovered a new way to do something, solved a problem that could be necessary later, save it as a skill with the skill tool."
```

核心区分：声明性知识 vs 程序性知识

维度	MEMORY.md / USER.md	Skills
知识类型	声明性（是什么）	程序性（怎么做）
典型内容	"这个项目用 Poetry 而非 pip"	"如何为这个项目添加新依赖：poetry add xxx"
更新频率	极低（跨月）	低（周级别，但补丁频繁）
结构化程度	非结构化条目（自然语言）	YAML frontmatter + Markdown 结构
容量上限	2200/1375 字符（严格限制）	无限制（可包含子文件）
复用方式	被动的上下文提示	主动的工具调用触发
可执行性	 仅供阅读	 包含步骤、命令、验证方法

何时选技能（skill_manage）：

1. 多步骤工作流：需要 5+ 工具调用才能完成的任务
2. 成功路径可复现：发现了一条解决某类问题的标准路径
3. 需要精确指令：包含具体命令、参数、验证步骤
4. 有陷阱需警告：这类任务有常见的坑需要避免
5. 跨平台差异：需要根据 OS 调整执行方式

示例场景：

- "如何在这个 monorepo 中添加一个新的微服务"
- "如何为这个项目写集成测试"
- "如何 debug 这套 CI 管道的超时问题"

何时选记忆（memory）：

1. 静态事实：环境配置、工具版本、约定俗成
2. 用户偏好：沟通风格、技术栈偏好、踩过的坑

3. 一次性发现：下次还能用但不值得固化完整流程

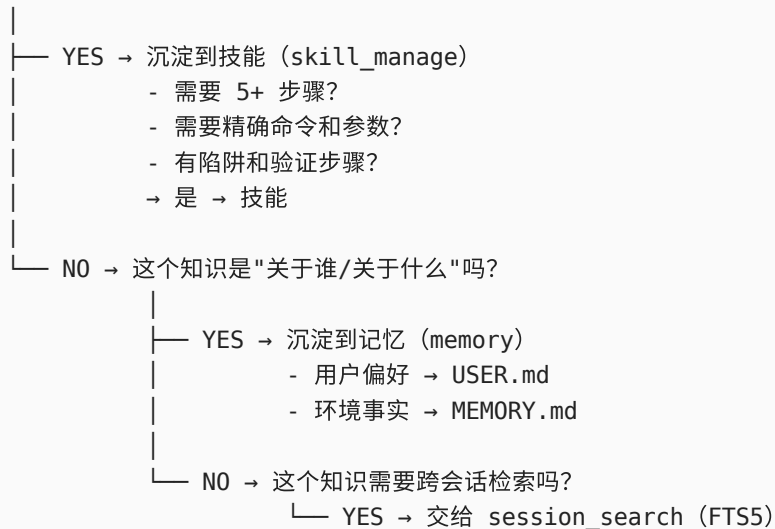
4. 配置信息：API endpoint、特殊环境变量

示例场景：

- "这个用户习惯用中文交流"
- "这个项目的 Python 版本是 3.11"
- "用户不喜欢被直接打断"

一个决策树 (Decision Tree)：

这个知识是"如何做某类任务"吗？



设计哲学的深层解读

1. 记忆是"轻量级缓存"，技能是"重量级索引"

- 记忆条目随手可写、字符受限，暗示"不要过度记忆"
- 技能有版本、平台、security scan，暗示"技能需要维护成本"

2. 技能的存在本身就是对任务复杂度的断言

- 如果你决定创建技能，你就是在说"这个问题值得标准化"
- 技能的创建门槛（需要写 SKILL.md）比记忆（一条 add 调用）高得多

3. 记忆和技能的互补性

- 技能可以引用记忆中的内容（如"参考 MEMORY.md 中的项目约定"）
- 记忆可以指向技能（如"这个项目的部署流程见 deploy-workflow 技能"）
- 两者不是替代关系，而是互补关系

3.2.4 反馈信号设计理念

🔥 Important

核心设计洞察：记忆的反馈信号本质上是"信息寿命预期"

Hermes Agent 的记忆系统并非被动存储，而是一套主动的信息生命周期管理系统。不同层级的记忆对应不同的反馈循环频率和信息保真度要求。

三层记忆的信号反馈特性对比：

记忆层	反馈信号来源	更新频率	信号类型	一致性机制
MEMORY.md	Agent自评估（LLM判断"这是否值得记住"）	极低（跨月）	隐式语义	冻结快照模式
SQLite会话	外部检索查询触发	中（周级别）	显式查询+FTS5	WAL并发控制
技能系统	技能执行结果+用户反馈	低（周级别）	隐式+显式混合	版本字段+快照缓存

设计哲学：

设计决策	原因
冻结快照模式（Frozen Snapshot）	系统Prompt稳定，但工具响应显示最新状态
字符限制（非Token限制）	模型无关性（不依赖具体模型的Tokenizer）
注入攻击扫描	防止用户注入恶意内容操纵Agent

"冻结快照"模式的设计哲学（Frozen Snapshot）

- 系统Prompt在会话期间保持稳定，但 memory 工具的每次响应都反映最新状态
- 这解决了LLM推理的**一致性-新鲜度矛盾**：推理需要稳定的上下文，但现实是持续变化的
- 洞见：这实际上是一种**乐观缓存 + 延迟失效策略**——假设快照在短期内足够好，用工具调用获取增量更新
- 对比：另一种方案是每次都重新加载全部记忆，但会导致每次推理都付出不必要的上下文开销

字符限制（非Token限制）的深层原因

- Tokenization与模型绑定，不同模型tokenizer不同
- 但更深的设计意图是：**让记忆内容脱离模型依赖，成为真正的"持久化知识"**
- 这使得记忆可以在不同模型间迁移，也使得记忆管理工具更稳定

注入攻击扫描的反向信号价值

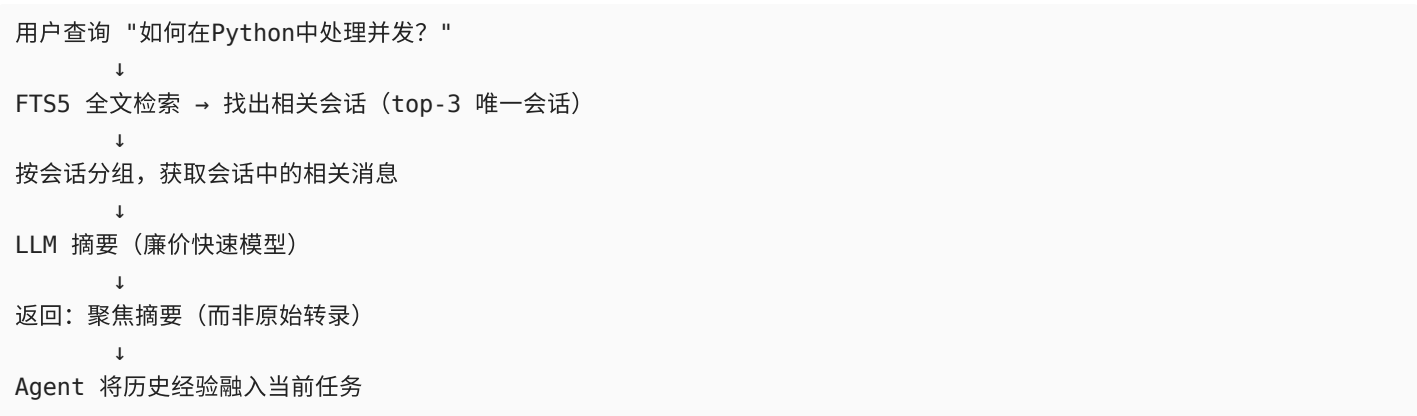
- 扫描本身也是一种"负反馈"信号：任何被阻止的内容，都是用户试图注入的操纵信号
- 这种被动检测信号，可以反过来帮助识别**高风险用户交互模式**（是否有人在持续尝试注入？）

3.3 跨会话检索与模式识别

文件: tools/session_search_tool.py

这是Hermes Agent实现"经验复用"的关键机制。

工作流程



与记忆系统的协同

检索目标	工具	存储位置
历史对话经验	session_search	SQLite FTS5
Agent自身笔记	memory	MEMORY.md
用户偏好	memory + honcho	USER.md + Honcho
结构化技能	skills / skill_manage	~/.hermes/skills/

3.4 上下文压缩与信息保护

文件: agent/context_compressor.py

压缩策略

```
# 保护区域（永不压缩）：
PROTECTED_HEAD = ["system", "first_human", "first_gpt", "first_tool"]

# 压缩区域：
# - 超过 token_budget 的中间消息
# - 按 LLM 摘要压缩
# - 保留足够的上下文用于工具调用

# 尾部保护：
# - 最后 ~20K tokens 保持原始（最新上下文最重要）
```

3.4.1 上下文压缩的反馈信号设计

🔥 Important

核心设计洞察：压缩的反馈信号是"信息可压缩性"而非"信息重要性"

上下文压缩的核心假设是：**中间轮次的历史信息，在经过摘要压缩后，仍能保留足够驱动正确推理的"信号"**。这是一个经验性假设，目前 Agent 产品的上下文压缩也通常采用该范式

三层保护机制的信号价值分析：

保护层级	保护内容	信号来源	设计逻辑
头部保护	system + first exchange	结构性位置	推理基础框架，一旦破坏则整个会话无意义
尾部保护	最近 ~20K tokens	动态Token预算	最新上下文直接影响当前推理质量
中间压缩	历史交互	LLM摘要质量	历史经验的"语义摘要"保留了模式而非细节

迭代摘要（self._previous_summary）的设计价值 -> 分段渐进式增量压缩方法

- 第一次压缩：从零生成摘要
- 第二次+压缩：基于前次摘要更新，而非重新总结
- 这与差分压缩的思想一致：增量变化只需要记录增量信息
- 避免了在多次压缩后会话历史被过度摘要（"摘要的摘要的摘要"）导致信号衰减

Structured Summary Template（Goal/Progress/Decisions/Files/Next Steps）的设计意图

- 参考了 Pi-mono 和 OpenCode 的经验
- 每个字段对应一种信息类型：目标（方向）、约束（偏好）、进展（状态）、决策（路径）、文件（资源）、下一步（计划）

- 洞见：这六元组实际上编码了agent任务执行的**完整MDP状态**（MDP = Markov Decision Process）

消息列表

↓
识别 PROTECTED_HEAD（系统级消息）
↓
识别 PROTECTED_TAIL（最近 ~20K tokens）
↓
中间区域 → LLM 摘要 → 单条摘要消息替换
↓
验证压缩后 $\text{token_count} \leq \text{token_budget}$
↓
如仍超限，递归压缩（再次摘要）

3.5 RL训练数据管道

Hermes Agent 内置完整的RL训练数据管道，**可将Agent经验转化为可训练的RL数据**。

核心设计洞察：Hermes Agent的RL管道是一个"验证-数据-训练"三层解耦架构，三层解耦架构的信号流动：

Layer 1: 轨迹收集 (HermesAgentLoop)

- 输出: AgentResult(messages, turns_used, tool_errors, reasoning)
- 反馈信号: 工具调用序列 + 错误记录（结构化的中间过程数据）

Layer 2: 奖励计算（子类 compute_reward + ToolContext）

- 输入: AgentResult + ToolContext（完整工具访问权限）
- 输出: float reward（通常 0.0-1.0，但任意浮点有效）
- 反馈信号: 由子类定义（奖励函数可以是任何东西：代码正确性、用户满意度、或自定义指标）——这是最关键的信号来源

Layer 3: 训练数据压缩 (TrajectoryCompressor)

- 输入: 原始轨迹 + reward
- 输出: 压缩后轨迹（ShareGPT格式）
- 合规约束: 训练框架上下文窗口上限（15,250 tokens）——超出会塞不进训练器，这是硬性要求

Note

注意：Layer 3没有用到 ContextCompressor

轨迹压缩不是"会话中的上下文压缩"，它们是两个完全独立的模块：

- agent/context_compressor.py：会话进行中实时触发，作用于 {role/content} 格式，服务于**推理**
- trajectory_compressor.py：会话结束后离线批量处理，作用于 **ShareGPT JSONL**，服务于**训练数据生成**

唯一共同点是都调用了 call_llm() 走 LLM 做摘要，但就连 prompt 模板都不一样——**前者输出结构化六元组，后者输出自然语言描述**。

ToolContext 作为奖励函数的"万能接口"

- compute_reward() 获得的是完整的工具访问权限（terminal, file, web, browser...）
- 这意味着奖励函数可以**直接执行真实验证**：不仅看Agent说了什么，还验证它做对了什么
- 例：检查文件是否真的被创建、命令是否真的成功运行、测试是否真的通过
- 洞见：这本质上是 **Ground Truth 验证**，而非LLM自评——奖励信号来自客观世界，而非Agent的自我感知

Note

RL管道的创新在于**奖励函数与训练框架的完全解耦**：`compute_reward()` 是抽象方法，由子类实现，这意味着**奖励函数可以是任何东西：代码正确性、用户满意度、或自定义指标**。

两阶段（Phase 1 / Phase 2）设计的信号质量区别

阶段	服务器类型	Token追踪	适用场景	训练信号质量
Phase 1	OpenAI兼容	无（占位符tokens）	SFT数据生成、验证测试	不能用于RL（缺logprobs）
Phase 2	VLLM/SGLang ManagedServer	精确token IDs + logprobs	RL训练（GRPO/PPO）	完整（可用于 advantage estimation）

- Phase 1的轨迹数据有messages但没有精确的token级信号，不能计算优势估计
- Phase 2是真正的RL训练模式，但需要VLLM服务器基础设施
- 这种分阶段设计允许**先收集数据（Phase 1），再升级到训练（Phase 2）**，降低了冷启动门槛

3.5.1 轨迹保存

文件: `agent/trajectory.py` / `trajectory_compressor.py`

轨迹（Trajectory）是Agent与环境交互的完整序列：

```
{
  "session_id": "xxx",
  "model": "claude-sonnet-4-6",
  "trajectory": [
    {"role": "user", "content": "帮我重构这个模块..."},
    {"role": "assistant", "content": "...", "tool_calls": [...]},
    {"role": "tool", "tool_call_id": "...", "content": "..."},
    {"role": "assistant", "content": "好的，我开始重构..."},
  ],
  "success": true,
  "tool_count": 7,
  "created_at": 1712000000
}
```

3.5.2 轨迹压缩——训练信号的"有损压缩"

Important

核心设计洞察：对于训练轨迹的压缩，其质量取决于对**"训练信号在哪里"**的理解

轨迹压缩与上下文压缩不同：后者服务于推理质量，前者服务于训练信号的质量。这意味着判断**"什么该压缩、什么该保留"**时，标准不是**"这对当前任务重要吗"**，而是**"这对模型学习重要吗"**

训练信号的三优先级模型：

优先级	内容	理由
P0 必须保留	system prompt + first human + first gpt + first tool	行为模式学习的"锚点"，决定了策略的初始方向

优先级	内容	理由
P1 必须保留	last 4 turns	最终状态和结论直接影响 reward，尾部包含最清晰的因果信号
P2 可压缩	中间所有轮次	只要摘要能保留关键决策路径，中间过程的细节可被压缩

为什么摘要策略是"描述性总结"而非"推理过程提取"？

- 摘要prompt要求：描述assistant做了什么（工具调用、搜索、文件操作），而非它想了什么
- 理由：训练数据中，模型的工具调用序列才是需要学习的行为，而非推理链
- 这与 reasoning_per_turn 的设计形成对比：推理内容被单独提取用于显示（wandb），但不被注入训练数据

压缩指标的元价值：信号本身就是数据

- TrajectoryMetrics 记录：原始tokens、压缩后tokens、压缩率、API调用次数、错误数
- 这些指标不只是"日志"，而是压缩质量的隐式反馈信号
- 例：still_over_limit: true 意味着压缩不够激进，下次需要调整 target_max_tokens
- compression_ratio 分布（min/max/median）是判断压缩策略是否稳定的依据

3.5.3 Atropos RL环境集成

文件: environments/hermes_base_env.py / environments/agent_loop.py / tools/rl_training_tool.py

与 Tinker-Atropos RL框架的集成：

```
# 两种运行模式：
# Phase 1: OpenAI 兼容服务器
# Phase 2: vLLM ManagedServer (GPU加速推理)

# 奖励函数设计：
class ToolContext:
    """提供每个rollout的工具访问，用于自定义奖励计算"""
    get_tool_calls()      # 获取工具调用序列
    get_rewards()         # 计算奖励信号
    get_trajectory()      # 获取完整轨迹
```

训练环境：

环境	用途
HermesSweEnv	SWE-bench风格软件工程任务
TerminalBench2EvalEnv	终端基准测试评估
TerminalTestEnv	栈验证测试

4. 辅助进化子系统

4.1 用户建模（Honcho）

文件: tools/honcho_tools.py / honcho_integration/session.py

🔗 Important

核心设计洞察：Honcho的"辩证式理解"本质上是将用户反馈信号蒸馏为结构化先验

Honcho不存储用户的"说过什么"，而是存储"理解了什么"——这是一个从原始信号到抽象表示的蒸馏过程（或者说从交互历史中进行隐性知识挖掘的过程）。这种设计解决了一个根本问题：用户不会主动告诉你他们是谁，他们只会告

告诉你他们要什么。

Honcho是 Nous Research 的AI原生用户建模系统（专门存"关于用户"的知识例如偏好、风格、习惯），提供超越简单键值存储的**dialectic（辩证）用户理解**：

工具	功能
honcho_profile	检索/更新用户精炼档案（peer card）
honcho_search	语义搜索存储的用户上下文
honcho_context	LLM驱动的辩证问答（理解用户意图）
honcho_conclude	将结论写回Honcho长期记忆

 **Note**

Honcho 的存储介质：外部服务（HTTP API）

与 SQLite 和文件系统不同，Honcho 的数据存储在 Nous Research 的 Honcho 服务端，本地仅维护一个消息缓存（`honcho_integration/session.py`），通过 HTTP API 与服务同步。这种设计**使得用户画像天然跨设备、跨会话共享**——换一台机器也能读到同一份档案。

代价是：依赖网络可用性、服务端数据不可本地直接编辑、以及与本地 `MEMORY.md / USER.md` 之间的一致性需要靠 `honcho_conclude` 手动同步。

用户建模流程：



4.2 多实例Profile隔离

文件: `hermes_cli/profiles.py`

Hermes Agent支持多实例Profile，完全隔离的运行环境：

```
~/hermes/
├─ config.yaml           # default profile
├─ .env
├─ memories/
├─ skills/
├─ sessions.db
├─ profiles/
│   └─ work/             # 工作场景profile
│       ├── config.yaml
│       ├── .env
│       └─ memories/
```

```
| | | skills/
| | | sessions.db
| | research/      # 研究场景profile
| | | ...
| | personal/
| | | ...
```

隔离内容：

- 配置文件（config.yaml）
- API密钥（.env）
- 记忆文件（memories/）
- 技能集合（skills/）
- 会话历史（sessions.db）
- 网关配置（gateway/）
- 定时任务（cron/）

Note

Honcho 在 Hermes 里的设计是面向用户（人）的，不是面向 Profile 的：

- Profile 是"工作场景隔离"
- Honcho 是"同一个人的认知共享"

所以如果用 work Profile 说了很多工作偏好，切换到 personal Profile 后，Honcho 依然记得这些——跨 Profile 全共享

4.3 MCP动态工具发现

文件: tools/mcp_tool.py

MCP（Model Context Protocol）支持使Agent能够**动态扩展工具能力**：

```
# MCP工具注册流程
mcp.discover_servers()      # 发现可用的MCP服务器
    ↓
mcp.list_tools()            # 获取工具列表
    ↓
mcp.call_tool(name, args)   # 调用工具
    ↓
notifications/tools/list_changed # 监听工具变化
    ↓
动态更新Agent工具集
```

5. 工具层自进化支持

工具注册表

文件: tools/registry.py

统一的工具注册中心，支持工具的发现、注册、权限控制和分发：

```
# — 工具注册中心 registry —————
# 每个工具模块在 import 时调用 register()，将自身声明到全局注册表
```

```

# model_tools.py 只查注册表，不再维护自己的工具列表
# -----

registry.register(
    name="skill_manage",          # 工具的唯一标识符，dispatch 时按名查找
    toolset="skills",            # 工具归属的工具集，用于按工具集批量启用/禁用

    # — OpenAI tool_calls 格式的 schema -----
    # 告诉 LLM: 这个工具叫什么、做什么、接受什么参数
    schema={
        "name": "skill_manage",    # schema 内必须与 name 字段一致
        "description": "Create, edit, or patch skills...",
        "parameters": {
            "type": "object",
            "properties": {
                "action": {
                    "type": "string",
                    "enum": ["create", "edit", "patch", "delete"]
                    # action 决定操作类型:
                    #   create   — 从成功经验新建技能
                    #   edit    — 完整重写技能内容
                    #   patch   — 快速补丁（针对特定错误/遗漏）
                    #   delete — 删除废弃技能
                },
                ...
            }
        }
    },

    # — 工具的实际执行逻辑 -----
    # 真正干活的函数，registry.dispatch(name) 时被调用
    # 可以是普通函数，也可以是 async 函数（由 is_async 标记）
    handler=lambda args, **kw: skill_manager_tool(...),

    # — 可用性检查函数 -----
    # dispatch 前会先调用 check_fn(), 返回 True 才将工具暴露给 LLM
    # 返回 False = 工具对 LLM 不可见（而不是报错）
    # check_fn 可以检查：环境变量、依赖是否安装、API key 是否配置等
    # skill_manage 需要 HERMES_SKILLS_DIR 存在，否则技能无法创建
    check_fn=check_requirements,

    # — 依赖的环境变量列表 -----
    # 不是强制约束（check_fn 才是），而是元数据
    # 用于提示/文档：告知这个工具依赖哪些环境变量
    # MCP 动态发现时会检查这些，确保工具可以正常运行
    requires_env=["HERMES_SKILLS_DIR"],
)

```

工具分类（Toolsets）

文件: toolsets.py

```

TOOLSETS = {
    "web":      ["web_tools"],
    "search":   ["web_tools"],

```

```

"vision": ["vision_tool"],
"image_gen": ["image_generation_tool"],
"terminal": ["terminal_tool", "environments"],
"skills": ["skills_tool", "skill_manager_tool"],
"browser": ["browser_tool"],
"cronjob": ["cron_tool"],
"messaging": ["messaging_tools"],
"rl": ["rl_training_tool", "trajectory_compressor"],
"file": ["file_tools"],
"tts": ["tts_tool"],
"todo": ["todo_tool"],
"memory": ["memory_tool"],
"session_search": ["session_search_tool"],
"clarify": ["clarification_tool"],
"code_execution": ["code_execution_tool"],
"delegation": ["delegate_tool"],
"honcho": ["honcho_tools"],
"homeassistant": ["homeassistant_tool"],
}

```

6. 轨迹工程：数据驱动的进化

完整轨迹生命周期

```

Step 1: 会话执行
    run_agent.py / batch_runner.py
    ↓
Step 2: 轨迹收集
    agent/trajectory.py → JSONL 文件
    (每个会话的完整交互历史)
    ↓
Step 3: 批量处理
    batch_runner.py → ShareGPT 格式
    (并行处理多个提示，统计工具使用)
    ↓
Step 4: 轨迹压缩
    trajectory_compressor.py
    (保护训练信号，压缩中间历史)
    ↓
Step 5: RL训练
    rl_cli.py + environments/ + Atropos
    (生成可训练的RL数据 or SFT数据)
    ↓
Step 6: 模型更新
    新版模型替代旧版
    ↓
Step 7: 闭环验证
    新模型在任务上表现更好
    → 产生更好的轨迹 → 更好的数据 → ...

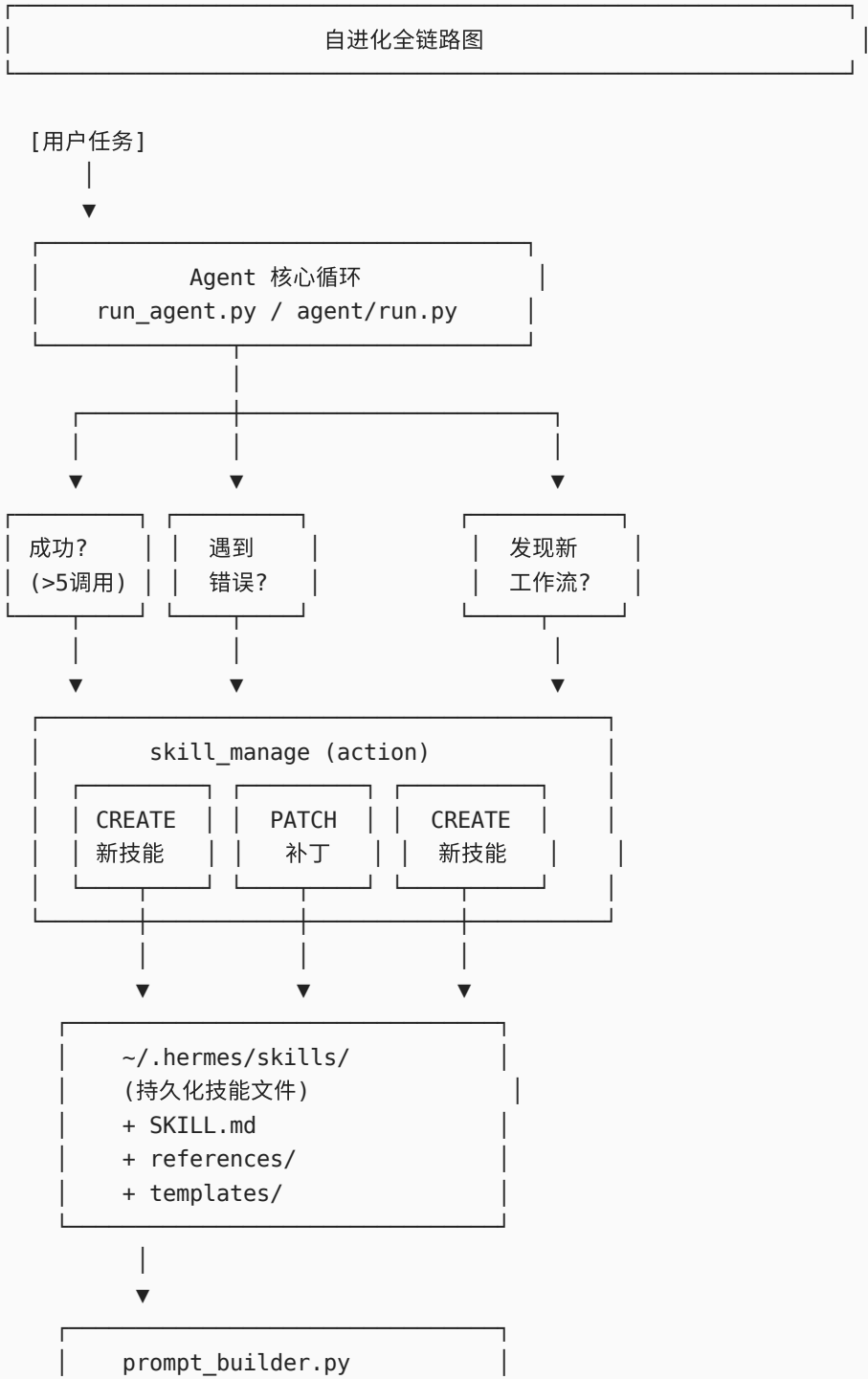
```

Batch Runner

文件: batch_runner.py

```
# 批量运行模式
batch_runner.run(
    prompts_file="prompts.jsonl",
    num_parallel=8,          # 并行数
    model="claude-sonnet-4-6",
    save_trajectories=True,
    tool_stats=True,         # 工具使用统计
    checkpoint_dir="./checkpoints",
)
```

7. 进化机制关联图



(技能自动注入系统Prompt)



Agent 下次遇到类似任务
→ 自动激活相关技能

记忆层 (跨会话持久化)

MEMORY.md — Agent笔记 — 环境事实、项目约定
USER.md — 用户画像 — 偏好、风格、期望
Honcho — 语义记忆 — 辩证理解、长期上下文
SQLite — 会话历史 — FTS5全文检索+LLM摘要



session_search 工具
(跨会话经验复用)

RL训练管道 (模型级进化)

trajectory.py — 轨迹收集
↓
batch_runner.py — 批量处理+统计
↓
trajectory_compressor.py — 训练数据压缩
↓
rl_cli.py + Atropos — RL训练
↓
新版模型 → 更好的Agent → 更好的轨迹 → ...

8. 技术实现细节

8.1 技能激活机制

技能通过 `prompt_builder.py` 的 `SKILLS_GUIDANCE` 注入系统Prompt:

```
SKILLS_GUIDANCE = """
After completing a complex task (5+ tool calls), fixing a tricky error,
or discovering a non-trivial workflow, save the approach as a
skill with skill_manage so you can reuse it next time.

When using a skill and finding it outdated, incomplete, or wrong,
patch it immediately with skill_manage(action='patch') - don't wait to be asked.
Skills that aren't maintained become liabilities.
"""
```

这意味着技能激活是通过**LLM的推理能力**而非硬编码规则实现的——Agent根据当前任务上下文，自主决定是否创建/使用/补丁技能。

8.2 记忆注入策略

记忆通过 `memory_tool.py` 的"冻结快照"模式注入：

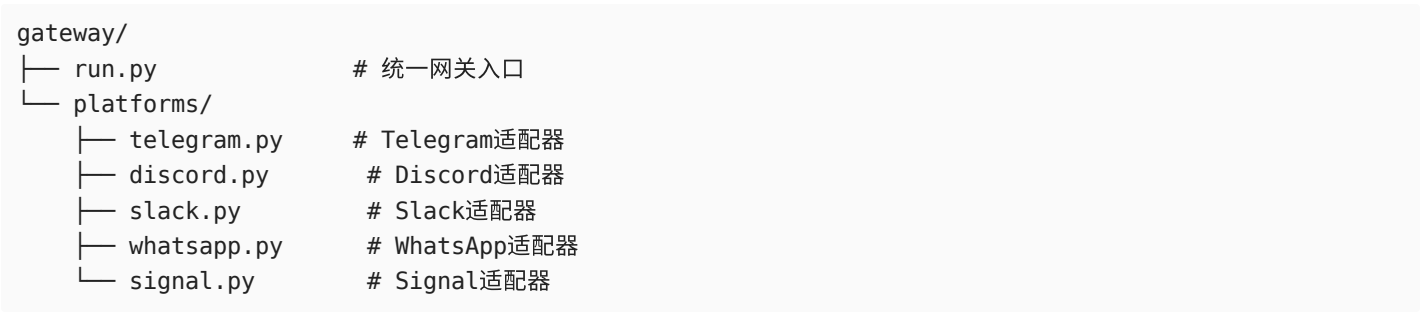


8.3 安全机制

安全措施	实现位置	说明
注入攻击扫描	<code>memory_tool.py</code>	扫描 <code>MEMORY.md</code> / <code>USER.md</code> 中的注入模式
技能安全扫描	<code>tools/skills_hub.py</code>	安装前校验技能内容
技能隔离	<code>~/.hermes/skills/</code>	技能在独立目录，避免相互影响
技能隔离 (Hub)	quarantine机制	未信任技能进入隔离区

8.4 平台适配层

文件: `gateway/platforms/`



每种消息平台的会话统一存储到 SQLite，会话检索可跨平台。

9. 总结与评价

9.1 核心设计哲学

Hermes Agent 的自进化机制遵循 **"安全进化"原则**——不修改Agent自身代码，而是通过三层持久化知识系统实现能力的累积和复用：

经验 → 技能（持久化知识）→ 跨会话复用 → 更好表现

↓

轨迹 → 训练数据 → 模型更新 → 更强能力

9.2 进化机制分类

进化层次	具体机制	可逆性	生效范围
技能层	创建/补丁技能	高（可编辑/删除）	特定任务类型
记忆层	MEMORY.md / USER.md	高（可编辑）	Agent行为风格
会话检索层	FTS5搜索 + LLM摘要	中（覆盖旧会话）	上下文理解
用户建模层	Honcho档案	中（持续更新）	个性化交互
轨迹/RL层	轨迹压缩 + RL训练	低（模型更新）	通用能力

9.3 创新点

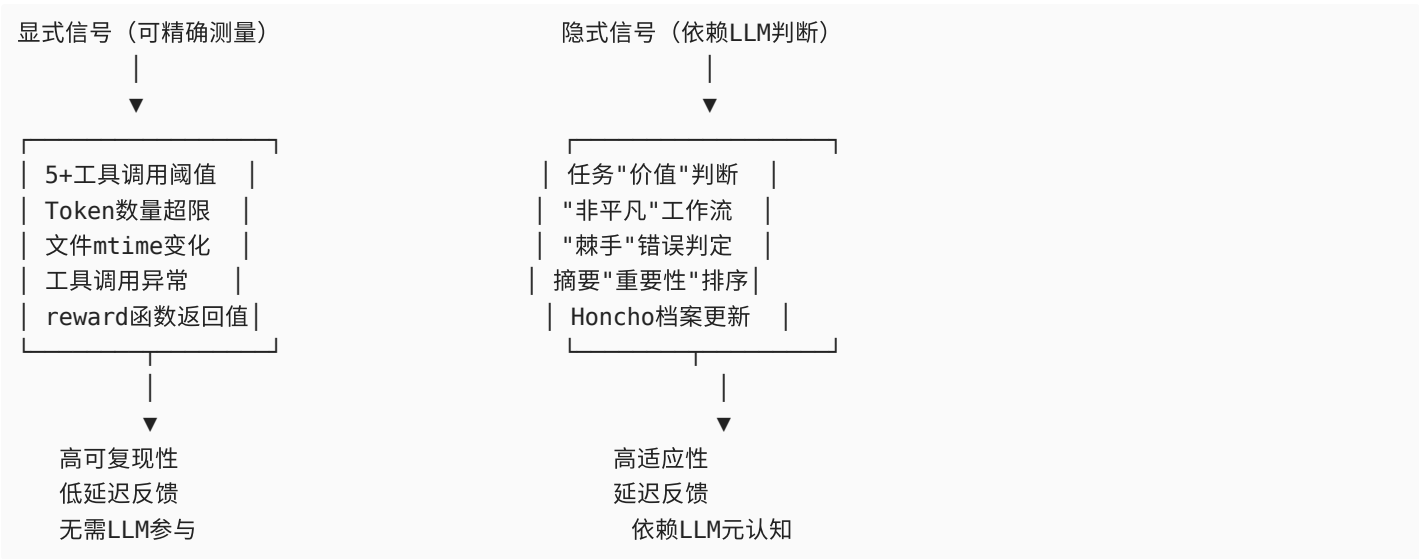
- 1. 技能作为一等公民：技能不是简单的提示词片段，而是有版本、平台适配、文件结构、引用子文档的完整知识模块
- 2. 冻结快照模式：系统Prompt稳定但记忆工具响应动态，解决了一致性和新鲜度的矛盾
- 3. FTS5 + LLM双层检索：全文索引保证速度，LLM摘要保证相关性
- 4. Dialectic用户建模：Honcho不是键值存储，而是"辩证"理解用户意图的系统
- 5. 轨迹压缩保护训练信号：不是简单截断，而是保护关键训练信号的有语义压缩

9.4 反馈信号体系全景分析

🔗 Important

这是理解Hermes Agent自进化机制的核心框架：反馈信号，而非代码结构，才是决定进化方向和速度的关键。

反馈信号的"显式-隐式"光谱：



五种反馈信号类型及其在系统中的分布：

信号类型	典型实例	反馈延迟	在Hermes中的体现
结构型	工具调用次数、Token数量	极低（毫秒级）	技能创建阈值、压缩触发
结果型	任务成功/失败、工具错误	低（秒级）	RL奖励计算、工具错误追踪
社交型	用户纠正、用户满意度	中（分钟-天）	Honcho conclude、USER.md更新

信号类型	典型实例	反馈延迟	在Hermes中的体现
语义型	LLM自评估、模式识别	中（取决于LLM速度）	技能创建决策、摘要质量
环境型	mtime变化、文件存在性	低（文件系统级）	技能缓存失效

隐式信号的元认知困境：谁来验证验证者？

- 技能创建依赖Agent判断"这是否值得记住"——但Agent的判断本身受限于它的认知能力
- 如果Agent**错误地低估**了某次经验的价值，它根本不会创建技能 → 进化停滞
- 如果Agent**错误地高估**了某次经验的价值，它创建了噪音技能 → 技能库退化
- 这是一个**自我指涉的反馈回路**：信号质量取决于系统自身，系统自身又受信号影响

最优雅的设计：工具调用成对完整性（_sanitize_tool_pairs）

- 这是一种**隐式约束反馈**：不直接说"这个动作是错的"，而是说"这个动作的返回值消失了"
- 本质上是用**结构完整性**而非**内容正确性**来检测问题
- 对比RL：这相当于"有效动作空间约束"——Agent可以犯任何错误，但不能破坏MDP的状态转移契约

9.5 局限性

1. **无代码级自我修改**：进化局限于提示词、记忆和技能，不修改底层逻辑
2. **技能质量依赖Agent判断**：如果Agent对"何时创建技能"判断错误，可能产生噪音技能
3. **RL训练管道需要外部Atropos**：自身只提供数据，不包含完整RL训练实现
4. **无显式进化评估机制**：没有量化指标衡量"进化"的程度或速度
5. **跨Profile技能不共享**：不同Profile的技能完全隔离，无法继承

参考文件索引

文件	作用
tools/skill_manager_tool.py	技能CRUD核心实现
tools/memory_tool.py	文件记忆系统
tools/session_search_tool.py	跨会话FTS5检索
tools/honcho_tools.py	Honcho用户建模接口
agent/prompt_builder.py	系统Prompt构建与技能注入
agent/context_compressor.py	上下文窗口压缩
agent/trajectory.py	轨迹收集
trajectory_compressor.py	训练数据压缩
batch_runner.py	批量轨迹收集
rl_cli.py	RL训练CLI
environments/hermes_base_env.py	Atropos RL环境
hermes_state.py	SQLite会话存储
hermes_cli/profiles.py	多Profile隔离
toolsets.py	工具集定义
tools/registry.py	工具注册表